



• ערכת לימוד מלאה • הכנה לראיונות הייטק

מדריך אלגוריתמים לראיונות הייטק

סיבוכיות • 8 התבניות • זיהוי דפוסים – הכל במקום אחד

שלושה חלקים: קודם **מבנים סיבוכיות** מאפס, אחר כך **8 התבניות הנפוצות** עם 24 שאלות פתורות, ולבסוף **16 חידות זיהוי** ללא שם האלגוריתם.
כל הקוד תקין ומוכן להרצה. בנוי לזיהוי מהיר של דפוסים – כי זה מה שבאמת נבדק.

תוכן העניינים

חלק א · סיבוכיות – המדריך למתחיל

- מה זה בכלל "סיבוכיות"?
- סימון Big-O ושני הכללים
- רמות הסיבוכיות הנפוצות
- איך מחשבים בפועל
- סיבוכיות זיכרון והטרייד-אוף

חלק ב · 8 התבניות הנפוצות (שאלות פתורות 24)

- 1 · שני מצביעים Two Pointers
- 2 · חלון מחליק Sliding Window
- 3 · מפת גיבוב Hash Map / Set
- 4 · סריקת גרפים ועצים BFS / DFS
- 5 · תכנון דינמי Dynamic Programming
- 6 · ערימה / תור עדיפויות Heap
- 7 · חיפוש בינארי על התשובה Binary Search
- 8 · מחסנית מונוטונית Monotonic Stack

חלק ג · זהה את התבנית (חידות 16)

- טבלת זיהוי מהיר – מילות המפתח

טיפים אחרונים לראיון

כל שורה בתוכן העניינים היא קישור – לחיצה עליה תקפיץ אותך ישירות לאזור המתאים במסמך.

סיבוכיות – המדריך למתחיל

Time & Space Complexity

מה זה בכלל "סיבוכיות"?

כשאומרים שאלגוריתם "יעיל" – לעומת מה? קוד שעובד מצוין על 10 פריטים עלול לקרוס על מיליון. סיבוכיות מתארת איך זמן הריצה (או הזיכרון) גדל ככל שהקלט גדל.

אנלוגיה: לחפש חבר בבניין. לדפוק על כל דלת = ככל שיש יותר דירות, החיפוש ארוך יותר. לוח שמות בכניסה שמפנה ישר לדירה = אותו זמן בלי קשר לגודל. שני אלגוריתמים, סיבוכיות שונה לחלוטין.

הנקודה: לא מודדים שניות (תלוי במחשב), אלא שואלים – אם הקלט יוכפל, מה יקרה לזמן? יישאר אותו דבר? יוכפל? יתחלף פי-ארבע?

סימון Big-O ושני הכללים

הסימון נכתב $O(...)$, כש- n מייצג את גודל הקלט. שני כללי אצבע:

כלל 1 – מתעלמים מקבועים: $2n$ או $100n$ שניהם $O(n)$. מעניינת הצורה (קו ישר), לא השיפוע.
כלל 2 – שומרים את הדומיננטי: $n^2 + n$ נכתב $O(n^2)$. כשהקלט עצום, ה- n^2 מגמד את השאר.

רמות הסיבוכיות הנפוצות

סימון	שם	מה זה אומר	פעולות ל- n =מיליון
$O(1)$	קבוע	זמן קבוע ללא קשר לקלט	1
$O(\log n)$	לוגריתמי	חוצים את הבעיה בכל צעד	~20
$O(n)$	לינארי	עוברים על כל פריט פעם אחת	1,000,000
$O(n \log n)$	לינארי-לוגריתמי	מיון יעיל, "חלק וכבוש"	מיליון ~20
$O(n^2)$	ריבועי	לולאה בתוך לולאה	טריליון
$O(2^n)$	מעריכי	כל צירוף אפשרי	אסטרונומי

ההבדל המטורף: למיליון פריטים, $O(n)$ עושה מיליון פעולות (חלקיק שנייה). $O(n^2)$ עושה טריליון – שעות או ימים. אותה בעיה, תהום ביניהן.

איך מחשבים בפועל

ספור לולאות: לולאה אחת = $O(n)$, לולאה בתוך לולאה = $O(n^2)$. **חיתוך לחצי מוסיף $\log n$.** פעולות עוקבות – מחברים ושומרים את הגדולה. מקוננות – מכפילים.

```
def example(arr):  
    # n = len(arr)  
    total = 0 # O(1)  
    for x in arr: total += x # O(n)  
    for i in arr: # לנל"א  
        for j in arr: print(i, j) # ... בתוך לולאה = O(n^2)  
    return total  
# הדומיננטי מנצח - O(1)+O(n)+O(n^2) = O(n^2) : סה"כ
```

סיבוכיות זיכרון והטרייד-אוף

אותו סימון, אבל לזיכרון נוסף. כמה משתנים בודדים = $O(1)$; מערך חדש בגודל הקלט = $O(n)$.

הטרייד-אוף הקלאסי: אפשר "לקנות" זמן בעזרת זיכרון. ב-Two Sum (בהמשך) נשתמש ב-hash map (זיכרון $O(n)$) כדי לחתוך זמן מ- $O(n^2)$ ל- $O(n)$.

מקרה גרוע מול ממוצע: שליפה מ-hash map היא $O(1)$ בממוצע, אבל במקרה גרוע נדיר $O(n)$. אזכור הניואנס מראה הבנה עמוקה.

8 התבניות הנפוצות

לכל תבנית: הסבר, שימוש פרקטי, "מתי לזהות", ושלוש שאלות פתורות עם קוד וניתוח סיבוכיות.

01 שני מצביעים (Two Pointers)

שני אינדקסים שרצים על אותו מבנה, בדרך כלל אחד מכל קצה. במקום שתי לולאות מקוננות ($O(n^2)$) – מעבר אחד ($O(n)$).

שימוש פרקטי: השוואת מחרוזות, מיזוג נתונים ממוינים, פלינדרום, איתור זוגות שמקיימים תנאי.

מתי לזהות: המערך ממין, ומחפשים זוג/שלישייה שמקיימים תנאי על הסכום – או השוואת קצוות אל המרכז.

Q1 מערך ממוין ו-target. החזר אינדקסים של שני מספרים שמסתכמים ל-target.

מצביע בהתחלה ובסוף. סכום גדול מדי → מזיזים ימני שמאלה; קטן מדי → שמאלי ימינה.

```
def two_sum(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        s = nums[lo] + nums[hi]
        if s == target: return [lo, hi]
        elif s < target: lo += 1
        else: hi -= 1
```

זמן $O(n)$ · זיכרון $O(1)$

Q2 בדוק אם מחרוזת היא פלינדרום, תוך התעלמות מתווים לא-אלפאנומריים ומאותיות גדולות/קטנות.

מצביע מכל קצה, מדלגים על תווים לא-רלוונטיים ומשווים אל המרכז.

```
def is_palindrome(s):
    lo, hi = 0, len(s) - 1
    while lo < hi:
        while lo < hi and not s[lo].isalnum(): lo += 1
        while lo < hi and not s[hi].isalnum(): hi -= 1
        if s[lo].lower() != s[hi].lower(): return False
        lo += 1; hi -= 1
    return True
```

זמן $O(n)$ · זיכרון $O(1)$

Q3 מצא את כל השלישיות הייחודיות במערך שסכומן 0. (3Sum)

ממיינים, מקבעים מספר בלולאה, ועל היתר מפעילים two pointers. דילוג על כפילויות מונע חזרות.

```
def three_sum(nums):
    nums.sort(); res = []
    for i in range(len(nums)):
        if i > 0 and nums[i] == nums[i-1]: continue
        lo, hi = i + 1, len(nums) - 1
        while lo < hi:
            t = nums[i] + nums[lo] + nums[hi]
            if t < 0: lo += 1
            elif t > 0: hi -= 1
            else:
                res.append([nums[i], nums[lo], nums[hi]]); lo += 1
                while lo < hi and nums[lo] == nums[lo-1]: lo += 1
    return res
```

זמן $O(n^2)$ · זיכרון $O(1)$ (לעומת $O(n^3)$ נאיבי)

02 חלון מחליק (Sliding Window)

מחזיקים "חלון" רציף עם שני קצוות, מרחיבים ומכווצים לפי הצורך. כל איבר נכנס ויוצא לכל היותר פעם אחת. → $O(n)$

שימוש פרקטי: ניתוח זרמי נתונים, עיבוד אותות, קטע רציף אופטימלי, "הכי ארוך/קצר שמקיים תנאי".

מתי לזהות: "רציף" + "הכי ארוך/קצר/בדיוק K" על מערך או מחרוזת.

Q1 אורך תת-המחרוזת הארוכה ביותר ללא תווים חוזרים.

מרחיבים ימינה עם set. תו שכבר קיים → מכווצים משמאל עד שיוצא.

```
def longest_unique(s):
    seen = set(); lo = 0; best = 0
    for hi in range(len(s)):
        while s[hi] in seen:
            seen.remove(s[lo]); lo += 1
        seen.add(s[hi]); best = max(best, hi - lo + 1)
    return best
```

זמן $O(n)$ · זיכרון $O(\min(n, \text{אלפבית}))$

Q2 סכום מקסימלי של תת-מערך רציף בגודל קבוע K.

חלון קבוע: סכום K ראשונים, ואז מחליקים — מוסיפים נכנס, מחסירים יוצא.

```
def max_sum_k(nums, k):
    window = sum(nums[:k]); best = window
    for i in range(k, len(nums)):
        window += nums[i] - nums[i-k]
        best = max(best, window)
    return best
```

זמן $O(n)$ · זיכרון $O(1)$

Q3 מערך חיובי ו- $target$. אורך תת-המעריך הרציף הקצר ביותר שסכומו $\leq target$.

חלון משתנה: מרחיבים וצוברים; כשהסכום $\leq target$ מכווצים משמאל כל עוד התנאי נשמר.

```
def min_subarray(target, nums):
    lo = 0; total = 0; best = float('inf')
    for hi in range(len(nums)):
        total += nums[hi]
        while total >= target:
            best = min(best, hi - lo + 1)
            total -= nums[lo]; lo += 1
    return best if best != float('inf') else 0
```

זמן $O(n)$ · זיכרון $O(1)$

03 מפת גיבוב (Hash Map / Hash Set)

חיפוש, הוספה ומחיקה ב- **$O(1)$** ממוצע. הרעיון: "להחליף זמן בזיכרון" – לשמור מה שכבר ראינו כדי להימנע מסריקה חוזרת.

שימוש פרקטי: ספירת תדירויות, deduplication, אינדוקס וחיפוש מהיר, caching, זיהוי קבוצות.

מתי לזהות: "האם ראיתי כבר X?", "כמה פעמים מופיע Y?", או כשנאיבי דורש לולאה בתוך לולאה לחיפוש.

Q1 מערך (לא ממוין) ו- $target$. החזר אינדקסים של שני מספרים שסכומם $target$.

במעבר יחיד, לכל מספר בודקים אם המשלים כבר נראה. שומרים ערך $\rightarrow$ אינדקס.

```
def two_sum(nums, target):
    seen = {}
    for i, n in enumerate(nums):
        if target - n in seen:
            return [seen[target - n], i]
        seen[n] = i
    return []
```

זמן $O(n)$ · זיכרון $O(n)$ (לעומת $O(n^2)$ נאיבי)

Q2 קבץ יחד מילים שהן אנגרמות זו של זו.

אנגרמות זהות במיון אותיות. המחזרות הממוינת היא המפתח במפה.

```
from collections import defaultdict
def group_anagrams(words):
    groups = defaultdict(list)
    for w in words:
        groups[''.join(sorted(w))].append(w)
    return list(groups.values())
```

זמן $O(n \cdot k \log k)$ · זיכרון $O(n \cdot k)$

Q3 אורך הרצף העוקב הארוך ביותר במערך לא ממוין, ב- $O(n)$.

שמים הכל ב-set. רץ עוקב מתחיל רק במספר ש- $n-1$ אינו ב-set, ומשם סופרים קדימה.

```
def longest_consecutive(nums):
    s = set(nums); best = 0
    for n in s:
        if n - 1 not in s:
            length = 1
            while n + length in s: length += 1
            best = max(best, length)
    return best
```

זמן $O(n)$ · זיכרון $O(n)$

04 סריקת גרפים ועצים (BFS / DFS)

DFS צולל לעומק (רקורסיה/stack). **BFS** סורק שכבה-שכבה עם תור, ולכן מוצא מסלול קצר ביותר ללא משקלים.

שימוש פרקטי: רשתות חברתיות, ניווט ומפות, crawling, ניתוח תלויות, מטריצות (כל תא = צומת).

מתי לזהות: "מסלול קצר ביותר ללא משקלים" → BFS. "רכיבי קשירות / ספירת איים" → DFS. מטריצה = גרף.

Q1 מטריצה של '1' (יבשה) ו-'0' (מים). ספור כמה "איים" יש.

כשנתקלים ביבשה לא מבוקרת – מגדילים מונה ומפעילים DFS ש"מטביע" את כל היבשה המחוברת.

```
def num_islands(grid):
    rows, cols = len(grid), len(grid[0]); count = 0
    def dfs(r, c):
        if r < 0 or c < 0 or r >= rows or c >= cols or grid[r][c] != '1': return
        grid[r][c] = '0'
        dfs(r+1, c); dfs(r-1, c); dfs(r, c+1); dfs(r, c-1)
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1': count += 1; dfs(r, c)
    return count
```

זמן $O(\text{שורות} \cdot \text{עמודות})$

Q2 החזר ערכי עץ בינארי לפי שכבות (רמה-רמה).

BFS עם תור. בכל איטרציה מעבדים שכבה שלמה ומכניסים את הילדים לשכבה הבאה.

```
from collections import deque
def level_order(root):
    if not root: return []
    res = []; q = deque([root])
    while q:
        level = []
        for _ in range(len(q)):
            node = q.popleft(); level.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        res.append(level)
    return res
```

זמן $O(n)$ · זיכרון $O(n)$

Q3 תפוזים טריים (1) ורקובים (2). בכל דקה רקוב מרכיב שכנים. כמה דקות עד שהכל רקוב?

BFS מרובה-מקורות: מתחילים עם כל הרקובים בתור ומתקדמים שכבה-שכבה (כל שכבה = דקה).

```
def oranges_rotting(grid):
    q = deque(); fresh = 0
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c]==2: q.append((r,c))
            elif grid[r][c]==1: fresh += 1
    minutes = 0
    while q and fresh:
        minutes += 1
        for _ in range(len(q)):
            r,c = q.popleft()
            for dr,dc in [(1,0),(-1,0),(0,1),(0,-1)]:
                nr,nc = r+dr, c+dc
                if 0<=nr<len(grid) and 0<=nc<len(grid[0]) and grid[nr][nc]==1:
                    grid[nr][nc]=2; fresh-=1; q.append((nr,nc))
    return -1 if fresh else minutes
```

זמן $O(\text{שורות} \cdot \text{עמודות})$

05 תכנון דינמי (Dynamic Programming)

פירוק לתת-בעיות חופפות ושמירת תוצאות כדי לא לחשב פעמיים. קדיין הוא ה-DP הפשוט ביותר.

שימוש פרקטי: אופטימיזציה של משאבים, תזמון, השוואת מחרוזות/DNA, המלצות, מסחר (קדיין).

מתי לזהות: "כמה דרכים יש ל...", "המקסימום/מינימום של...", או כשרקורסיה נאיבית מחשבת אותה תת-בעיה שוב ושוב.

Q1 עולים 1 או 2 מדרגות בכל פעם. בכמה דרכים מגיעים למדרגה ה-n?

— פיבונאצ'י! שומרים שני ערכים אחרונים. $ways(n) = ways(n-1) + ways(n-2)$

```
def climb_stairs(n):
    a, b = 1, 1
    for _ in range(n): a, b = b, a + b
    return a
```

זמן $O(n)$ · זיכרון $O(1)$

Q2 גנב שודד בתים, אך לא שני סמוכים. מה הסכום המקסימלי?

. שומרים שני ערכים אחרונים. $rob(i) = \max(rob(i-1), rob(i-2) + nums[i])$

```
def rob(nums):
    prev, curr = 0, 0
    for n in nums:
        prev, curr = curr, max(curr, prev + n)
    return curr
```

זמן $O(n)$ · זיכרון $O(1)$

Q3 סוגי מטבעות וסכום. מספר המטבעות המינימלי שמרכיב את הסכום.

טבלה: $dp[x]$ = מינימום מטבעות לסכום x. לכל סכום מנסים כל מטבע.

```
def coin_change(coins, amount):
    dp = [float('inf')] * (amount + 1); dp[0] = 0
    for x in range(1, amount + 1):
        for c in coins:
            if c <= x: dp[x] = min(dp[x], dp[x - c] + 1)
    return dp[amount] if dp[amount] != float('inf') else -1
```

זמן $O(\text{amount} \cdot \text{מטבעות})$ · זיכרון $O(\text{amount})$

06 ערימה / תור עדיפויות (Heap / Priority Queue)

שליפת האיבר הקטן/גדול ביותר ב- $O(\log n)$ בלי למיין הכל. בפייתון `heapq` הוא min-heap (למקס מכניסים ערכים שליליים).

שימוש פרקטי: תורי משימות לפי עדיפות, K schedulers החשובים בזרם, מיזוג מקורות ממוינים, Dijkstra.

מתי לזהות: K הכי גדולים/קטנים/נפוצים, "החציון בזרם", או "תמיד צריך את המינימום/מקסימום הנוכחי".

Q1 מצא את האיבר ה-K בגודלו במערך לא ממור.

min-heap בגודל K. אם גדל מעבר ל-K, מסירים מינימום. השורש הוא ה-K בגודלו.

```
import heapq
def find_kth_largest(nums, k):
    heap = []
    for n in nums:
        heapq.heappush(heap, n)
        if len(heap) > k: heapq.heappop(heap)
    return heap[0]
```

זמן $O(n \log k)$ · זיכרון $O(k)$

Q2 החזר את K האיברים הנפוצים ביותר במערך.

סופרים תדירויות (hash map), ושולפים את K הגבוהים ביותר.

```
from collections import Counter
import heapq
def top_k_frequent(nums, k):
    count = Counter(nums)
    return heapq.nlargest(k, count.keys(), key=count.get)
```

זמן $O(n \log k)$ · זיכרון $O(n)$

Q3 מבנה שמקבל מספרים בזרם ומחזיר בכל רגע את החציון.

שתי ערימות: max-heap לחצי הקטן, min-heap לחצי הגדול, מאוזנות. החציון הוא הראש/ממוצע הראשים.

```
import heapq
class MedianFinder:
    def __init__(self): self.small = []; self.large = []
    def addNum(self, num):
        heapq.heappush(self.small, -num)
        heapq.heappush(self.large, -heapq.heappop(self.small))
        if len(self.large) > len(self.small):
            heapq.heappush(self.small, -heapq.heappop(self.large))
    def findMedian(self):
        if len(self.small) > len(self.large): return -self.small[0]
        return (-self.small[0] + self.large[0]) / 2
```

הוספה $O(\log n)$ · חציון $O(1)$

07 חיפוש בינארי על התשובה (Binary Search on the Answer)

חיפוש בינארי חוצה מערך ממור ב- $O(\log n)$. הטוויסט: מחפשים את התשובה עצמה בטווח, כל עוד יש פונקציה מונוטונית "האם זה אפשרי?".

שימוש פרקטי: תכנון קיבולת ("מהירות/הגודל המינימלי לעמוד בדליון"), אופטימיזציה, rate limiting.

מתי לזהות: "מצא את המינימום/מקסימום שעדיין מקיים תנאי", והתנאי מונוטוני.

Q1 חיפוש בינארי קלאסי: מצא target במערך ממין, או 1-

חוצים באמצע ומחליטים לאיזה חצי להמשיך. שווה לדעת בעל-פה.

```
def binary_search(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if nums[mid] == target: return mid
        elif nums[mid] < target: lo = mid + 1
        else: hi = mid - 1
    return -1
```

זמן $O(\log n)$ · זיכרון $O(1)$

Q2 קוקו אוכלת בננות מערימות תוך H שעות. המהירות המינימלית שתספיק?

טווח $max..1$ (ערימה). "מספיק" מונוטוני, אז מחפשים בינארית את המהירות המינימלית התקינה.

```
import math
def min_eating_speed(piles, h):
    def hours(speed):
        return sum(math.ceil(p / speed) for p in piles)
    lo, hi = 1, max(piles)
    while lo < hi:
        mid = (lo + hi) // 2
        if hours(mid) <= h: hi = mid
        else: lo = mid + 1
    return lo
```

זמן $O(n \log(\max))$ · זיכרון $O(1)$

Q3 חבילות במשקלים נתונים תוך D ימים בסדר נתון. קיבולת הספינה המינימלית?

טווח בין החבילה הכבדה לסכום הכל. לכל קיבולת בודקים כמה ימים, ומחפשים את המינימלית שעומדת ב-D.

```
def ship_within_days(weights, days):
    def days_needed(cap):
        d, load = 1, 0
        for w in weights:
            if load + w > cap: d += 1; load = 0
            load += w
        return d
    lo, hi = max(weights), sum(weights)
    while lo < hi:
        mid = (lo + hi) // 2
        if days_needed(mid) <= days: hi = mid
        else: lo = mid + 1
    return lo
```

זמן $O(n \log(\text{סכום}))$ · זיכרון $O(1)$

08 מחסנית מונוטונית (Monotonic Stack)

stack שאיבריו בסדר עולה/יורד עקבי. איבר שמפר את הסדר → מקפצים ומעבדים. בעיות $O(n^2)$ נפתרות ב- $O(n)$.

שימוש פרקטי: "האירוע הבא שגדול/קטן ממני", ניתוח מחירי מניות, עיבוד טווחים בהיסטוגרמות.

מתי לזהות: "האיבר הבא שגדול/קטן", "כמה ימים עד ש...", או כשנאיבי הוא "לכל איבר תסרוק קדימה".

Q1 בדוק אם מחרוזת סוגריים `{[]()}` תקינה ומאוזנת.

סוגר פותח נדחף; סוגר סוגר חייב להתאים לראש. בסוף – מחסנית ריקה = איזון.

```
def is_valid(s):
    pairs = {'(': ')', '[': ']', '{': '}'
    stack = []
    for ch in s:
        if ch in pairs:
            if not stack or stack.pop() != pairs[ch]: return False
        else: stack.append(ch)
    return not stack
```

זמן $O(n)$ · זיכרון $O(n)$

Q2 לכל יום, כמה ימים צריך לחכות לטמפרטורה חמה יותר?

מחסנית יורדת של אינדקסים. יום חם יותר הוא התשובה לימים שמחכים – מקפיצים וממלאים מרחק.

```
def daily_temperatures(temps):
    res = [0] * len(temps); stack = []
    for i, t in enumerate(temps):
        while stack and temps[stack[-1]] < t:
            j = stack.pop(); res[j] = i - j
        stack.append(i)
    return res
```

זמן $O(n)$ · זיכרון $O(n)$

Q3 גבהי עמודות בהיסטוגרמה. שטח המלבן הגדול ביותר שניתן לתחום.

מחסנית עולה של אינדקסים. עמודה נמוכה מקפיצה גבוהות; לכל אחת מחשבים מלבן. ה-0 בסוף מנקז.

```
def largest_rectangle(heights):
    stack = []; best = 0
    for i, h in enumerate(heights + [0]):
        while stack and heights[stack[-1]] > h:
            height = heights[stack.pop()]
            width = i if not stack else i - stack[-1] - 1
            best = max(best, height * width)
        stack.append(i)
    return best
```

זמן $O(n)$ · זיכרון $O(n)$ (לעומת $O(n^2)$ נאיבי)

16 חידות מעורבות, בלי שם האלגוריתם. קרא, נסה לזהות איזו תבנית מסתתרת, ואז בדוק מול קופסת התשובה. זו המיומנות האמיתית בראיון.

1 בקרת כפילויות במערכת תשלומים

רשימת מזהי עסקאות מהיום. קבע במהירות האם **עסקה כלשהי חזרה על עצמה** (חיוב כפול). הרשימה לא ממוינת וענקית.

התשובה

מפת גיבוב / Hash Set

הסימנים: "האם ראיתי כבר X?" + "במהירות" + לא ממיון. set נותן בדיקה ב- **$O(1)$** במקום סריקה חוזרת.

```
def has_duplicate(ids):
    seen = set()
    for tid in ids:
        if tid in seen: return True
        seen.add(tid)
    return False
```

זמן $O(n)$ · זיכרון $O(n)$ **2 מאגר המים הגדול ביותר**

שורת קירות אנכיים בגבהים שונים. בחר **שני קירות** שיחזיקו את כמות המים המקסימלית = המרחק × גובה הקיר הנמוך.

התשובה

שני מצביעים Two Pointers

הסימנים: בוחרים **זוג** מקצוות + אופטימום. מתחילים ברוחב מקסימלי ומזיזים פנימה את הנמוך – רק הוא יכול לשפר.

```
def max_area(heights):
    lo, hi = 0, len(heights) - 1; best = 0
    while lo < hi:
        h = min(heights[lo], heights[hi])
        best = max(best, h * (hi - lo))
        if heights[lo] < heights[hi]: lo += 1
        else: hi -= 1
    return best
```

זמן $O(n)$ · זיכרון $O(1)$

מסלול מדרגות, לכל אחת "עלות". אפשר 1 או 2 בכל צעד. מהי העלות הכוללת המינימלית לראש? (התחלה מ-0 או 1 בחינם).

התשובה

תכנון דינמי Dynamic Programming

הסימנים: "עלות מינימלית" + בחירה מצטברת (1/2) שמשפיעה על ההמשך. העלות למדרגה תלויה בשתיים שלפניה.

```
def min_cost(cost):
    a, b = 0, 0
    for i in range(2, len(cost) + 1):
        a, b = b, min(b + cost[i-1], a + cost[i-2])
    return b
```

זמן $O(n)$ · זיכרון $O(1)$

מחירי מניה יומיים. לכל יום, מצא את היום הקרוב הבא שבו המחיר גבוה יותר (אם אין -1).

התשובה

מחסנית מונוטונית Monotonic Stack

הסימנים: "האיבר הבא שגדול ממני" + נאיבי "לכל אחד תסרוק קדימה". מחסנית ימים שמחכים; יום גבוה הוא התשובה לכולם.

```
def next_higher_day(prices):
    res = [-1] * len(prices); stack = []
    for i, p in enumerate(prices):
        while stack and prices[stack[-1]] < p:
            res[stack.pop()] = i
        stack.append(i)
    return res
```

זמן $O(n)$ · זיכרון $O(n)$

חיישן מדווח ערך כל שנייה. מצא את חלון 5 השניות הרציף עם הממוצע הגבוה ביותר. (גודל קבוע).

התשובה

חלון מחליק (גודל קבוע) Fixed Sliding Window

הסימנים: "חלון רציף" + "גודל קבוע" + אופטימום. מחליקים: מוסיפים נכנס, מחסירים יוצא.

```
def max_avg(nums, k):  
    window = sum(nums[:k]); best = window  
    for i in range(k, len(nums)):  
        window += nums[i] - nums[i-k]  
        best = max(best, window)  
    return best / k
```

זמן $O(n)$ · זיכרון $O(1)$

כל חולה מקבל ציון חומרה. בכל רגע מטפלים **ביותר** שממתין. חולים מגיעים כל הזמן. תכנן מבנה ל"הוסף" ו"מי הבא".

התשובה

ערימה / תור עדיפויות Heap

הסימנים: "תמיד צריך את הקיצוני הנוכחי" + הכנסות בזמן אמת. ערימה: שליפה והוספה ב- $O(\log n)$.

```
import heapq  
class ER:  
    def __init__(self): self.h = []; self.t = 0  
    def admit(self, name, severity):  
        heapq.heappush(self.h, (-severity, self.t, name)); self.t += 1  
    def treat_next(self): return heapq.heappop(self.h)[2]
```

הוספה $O(\log n)$ · שליפה $O(\log n)$

בהינתן שני אנשים, מצא את מספר הקשרים המינימלי בשרשרת ההיכריות ביניהם. הקשרים אינם ממושקלים.

התשובה

סריקה לרוחב BFS

הסימנים: "מסלול קצר ביותר" + גרף ללא משקלים. BFS סורק שכבה-שכבה ומוצא מרחק מינימלי.

```
from collections import deque
def degrees(graph, start, target):
    if start == target: return 0
    visited = {start}; q = deque([(start, 0)])
    while q:
        person, d = q.popleft()
        for friend in graph[person]:
            if friend == target: return d + 1
            if friend not in visited:
                visited.add(friend); q.append((friend, d + 1))
    return -1
```

זמן O(צמתים + קשתות)

מערך שהיה ממזין אבל "סובב" בנקודה לא ידועה ([4,5,6,1,2,3]). מצא ערך ב- $O(\log n)$ – בלי לעבור על הכל.

התשובה

חיפוש בינארי (וריאציה) Modified Binary Search

הסימנים: הדרישה $O(\log n)$ + מערך ממזין (גם אם "שבור"). בכל חצי אחד הצדדים ממזין – מזהים ומחליטים לאן.

```
def search_rotated(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if nums[mid] == target: return mid
        if nums[lo] <= nums[mid]:
            if nums[lo] <= target < nums[mid]: hi = mid - 1
            else: lo = mid + 1
        else:
            if nums[mid] < target <= nums[hi]: lo = mid + 1
            else: hi = mid - 1
    return -1
```

זמן $O(\log n) \cdot O(1)$ זיכרון

שתי מילים. בדוק אם השנייה היא ערבוב מחדש של אותן אותיות בדיוק (אנגרמה).

התשובה

מפת גיבוב (ספירת תדירויות) Hash Map

הסימנים: "אותן אותיות, אותו מספר הופעות" = השוואת **תדירויות**. אנגרמות אם ורק אם מפות הספירה זהות.

```
from collections import Counter
def is_anagram(s, t):
    return Counter(s) == Counter(t)
```

זמן $O(n)$ · זיכרון $O(\text{אלפבית})$

מערך **ממוין** של מדידות. קבע אם קיימות **שתי מדידות** שההפרש ביניהן בדיוק **K**.

התשובה

שני מצביעים Two Pointers

הסימנים: מערך **ממוין** + זוג שמקיים תנאי על ההפרש. ההפרש קטן מדי → מרחיבים, גדול מדי → מכווצים.

```
def has_diff_k(arr, k):
    lo, hi = 0, 1
    while hi < len(arr):
        diff = arr[hi] - arr[lo]
        if diff == k and lo != hi: return True
        elif diff < k: hi += 1
        else:
            lo += 1
            if lo == hi: hi += 1
    return False
```

זמן $O(n)$ · זיכרון $O(1)$

מחרוזת ארוכה. מצא את אורך המקטע הרציף הארוך ביותר שמכיל לכל היותר 2 תווים שונים.

התשובה

חלון מחליק (גודל משתנה) Variable Sliding Window

הסימנים: "מקטע רציף הכי ארוך" + תנאי חוקיות. מרחיבים ימינה; מפר → מכווצים משמאל. מפת תדירות סופרת תווים שונים.

```
from collections import defaultdict
def longest_two(s):
    count = defaultdict(int); lo = 0; best = 0
    for hi in range(len(s)):
        count[s[hi]] += 1
        while len(count) > 2:
            count[s[lo]] -= 1
            if count[s[lo]] == 0: del count[s[lo]]
            lo += 1
        best = max(best, hi - lo + 1)
    return best
```

זמן $O(n)$ · זיכרון $O(1)$

סדרת מספרים. מצא את אורך תת-הסדרה העולה הארוכה ביותר – לא בהכרח רציפה (אפשר לדלג).

התשובה

תכנון דינמי Dynamic Programming

הסימנים: "הכי ארוך" + "לא בהכרח רציף" → לא חלון מחליק! $dp[i]$ = אורך הסדרה שמסתיימת ב- i .

```
def lis(nums):
    if not nums: return 0
    dp = [1] * len(nums)
    for i in range(len(nums)):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp)
```

זמן $O(n^2)$ · זיכרון $O(n)$

n חדרים נעולים מלבד חדר 0. בכל חדר מפתחות לחדרים אחרים. החל מחדר 0 – האם אפשר לבקר בכל החדרים?

התשובה

סריקה לעומק DFS

הסימנים: "האם אפשר להגיע לכל..." + צמתים (חדרים) וקשתות (מפתחות) = **קשירות**. DFS צולל, אוסף מפתחות, מסמן ביקורים.

```
def can_visit_all(rooms):
    visited = set(); stack = [0]
    while stack:
        room = stack.pop()
        if room in visited: continue
        visited.add(room)
        for key in rooms[room]:
            if key not in visited: stack.append(key)
    return len(visited) == len(rooms)
```

זמן $O(\text{חדרים} + \text{מפתחות})$

נקודות (לקוחות) על מפה. מצא את K הלקוחות הקרובים ביותר לחנות בראשית הצירים. אין צורך למיין את כולם.

התשובה

ערימה Heap

הסימנים: " K הכי קרובים/גדולים/נפוצים". ערימה בגודל K יעילה ממיון מלא. (אפשר להשוות מרחקים בריבוע).

```
import heapq
def k_closest(points, k):
    return heapq.nsmallest(
        k, points,
        key=lambda p: p[0]**2 + p[1]**2)
```

זמן $O(n \log k)$ • זיכרון $O(k)$

מחירי מניה יומיים. לכל יום, חשב כמה ימים רצופים אחורה (כולל היום) שבהם המחיר היה נמוך או שווה למחיר היום.

התשובה

מחסנית מונוטונית Monotonic Stack

הסימנים: "כמה ימים רצופים אחורה עד ש..." = חיפוש הגבול. מחסנית מחירים יורדים; המרחק לראש שנשאר הוא המוטה.

```
def stock_span(prices):
    res = [0] * len(prices); stack = []
    for i, p in enumerate(prices):
        while stack and prices[stack[-1]] <= p:
            stack.pop()
        res[i] = i + 1 if not stack else i - stack[-1]
        stack.append(i)
    return res
```

זמן $O(n)$ • זיכרון $O(n)$

תכנון קו הייצור

16

מספר מכונות; כל אחת מייצרת פריט כל t דקות (קצב שונה), במקביל. מהו הזמן המינימלי לייצר לפחות N פריטים?

התשובה

חיפוש בינארי על התשובה Binary Search on the Answer

הסימנים: "הזמן המינימלי לעמוד ביעד" + קל לבדוק "האם בזמן T מספיק?". מונוטוני, אז בינארי על טווח הזמן.

```
def min_time(machines, n):
    def produced(T):
        return sum(T // t for t in machines)
    lo, hi = 1, min(machines) * n
    while lo < hi:
        mid = (lo + hi) // 2
        if produced(mid) >= n: hi = mid
        else: lo = mid + 1
    return lo
```

זמן $O(\log(\text{טווח}))$ • (מכונות)

— טבלת זיהוי מהיר — מילות המפתח

חשוב על...

אם בשאלה מופיע...

Two Pointers
Sliding Window
Hash Map / Set
BFS
DFS
Dynamic Programming
Heap
Binary Search on Answer
Monotonic Stack

מערך ממזין + מצא זוג שמקיים תנאי
רציף + הכי ארוך/קצר / גודל קבוע
"האם ראיתי כבר?" / ספירת תדירויות / כפילות
מסלול קצר ביותר ללא משקלים / שכבות
קשירות / "אפשר להגיע ל..." / ספירת רכיבים
מקסימום/מינימום + בחירה מצטברת / לא רציף
"א הכי..." / תמיד צריך את הקיצוני הנוכחי
"המינימום שעדיין מקיים" + "האם X מספיק"
"האיבר הבא שגדול/קטן" / טווח אחורה

טיפים אחרונים לראיון

- נתח סיבוכיות בקול רם. "הנאיבי $O(n^2)$, עם hash map יורדים ל- $O(n)$ " – משפט כזה שווה זהב.
 - חשוב בקול רם. התחל מכוח גס, הסבר, ואז זקק. התהליך חשוב כמו התשובה.
 - שאל שאלות הבהרה. ממין? כפילויות? טווח הקלט? מה מחזירים אם אין תשובה?
 - בדוק מקרי קצה. מערך ריק, איבר בודד, כל הערכים שליליים (כמו בקדיין!), כפילויות.
 - תרגל NeetCode 150 ו-Blind 75. הן מכסות בדיוק את 8 התבניות האלה.
- הקשר ישראלי: בחברות הגדולות (גוגל/מטא/אמזון ת"א, אפל, אינטל) – LeetCode סטנדרטי. בסטארטאפים – לעיתים יותר מעשי, system design קטן או בעיה קרובה למוצר.

בהצלחה בראיון 🍀 · זכור: זיהוי התבנית < שינון פתרונות.